ICNNA Architectural Refactoring
Phase 2 Closing Minute (Session 4)

Steps E, F, and G — Transitional Strategy, Open Risks, and Session Documentation

Felipe Orihuela-Espina Claude (Computational Research Architect)

14 April 2026

Document Log:

- 14-Apr-2026: FOE/Claude — Document created. Records Phase 2 Steps E (transitional strategy), F (open risks registry), and G (session documentation and `claude/` folder reorganisation), formally closing Phase 2.

---

1 Session Overview

Date:
14 April 2026

Participants:
Felipe Orihuela-Espina (FOE), Claude Opus 4.6 (Computational Research Architect role)

Predecessors:
Phase 1 minute (`Phase1_minute.tex`); Phase 2a minute (12-Apr-2026); Phase 2b minute (13-Apr-2026); Phase 2c minute (`Phase2c_minute.tex`, 13-Apr-2026, Steps C and D).

Objective:
Complete Phase 2 Steps E (transitional strategy from the current dual state to the v2.0.0 endpoint; coexistence with Cantor; handover to Cantor; user communication), F (registry of open risks, unknowns, deferred decisions, and open questions), and G (session documentation and `claude/` folder reorganisation).

Outcome:
Phase 2 closed. Transitional strategy articulated as seven operating rules. Handover model defined with six policy axes. Registry of ∼30 open items established. `claude/` folder reorganised with a portable-by-default, memory-canonical structure. Three new policies codified (two in global CLAUDE.md, one in project context). Draft Phase 3 context prompt produced as handoff.

Model used:
Claude Opus 4.6

Status:
Phase 2 is formally closed with this minute. Phase 3 (roadmap definition) to follow in a separate session. FOE will prepare his own Phase 3 context prompt; the draft produced here serves as fallback and input.

1.1 Carried forward from Phase 2c

Phase 2c (13-Apr-2026) produced:

- The two-tier scope structure (survival tier + enhancement tier).
- Design principles P1–P9 for the bounded ICNNA update.
- The flat-architecture decision (`identifiableObject`-based IDs replace nested trees).
- The v2.0.0 endpoint definition (all classes in `+icnna`, flat architecture, new GUI late, BIDS compatibility, full test coverage, SNIRF maintained, complete documentation).
- The key clarification: v2.0.0 marks the end of *this modernisation plan*, not the end of ICNNA's journey.

These form the settled foundation on which Phase 2 Session 4 built. No Phase 2c decision was reopened.

2 Step E — Transitional Strategy

Step E was decomposed into four substeps, each addressed in turn. The decomposition is orthogonal: each substep concerns a distinct aspect of the transition and coexistence.

E.1 Internal transition: from the current `oo/ + +icnna` dual state to the v2.0.0 endpoint.

E.2 Coexistence with Cantor during Cantor's build-out.

E.3 Handover to Cantor.

E.4 User communication strategy across the three periods (modernisation, handover, post-handover).

2.1 E.1 — Internal transition

2.1.1 Framing

ICNNA at the start of this session is in a partial dual state: `src/oo/` carries ∼18 years of legacy MATLAB class-folder implementations; `src/+icnna/` carries ∼20 classes already modernised under the new package namespace. Both subsystems are in simultaneous live use. Phase 2c fixed the endpoint but not the transition trajectory. E.1 fills that gap by resolving five interlocking questions.

2.1.2 Q1 — Coexistence semantics per class

Decision: Soft cut. When modern `+icnna.X` ships, legacy `oo/@X` survives in deprecated form.
Rationale.

- **Consistency with established practice.** Soft cut is the long-standing pattern in ICNNA's own migration history. Changing it mid-stream creates asymmetry.
- **P3 compliance.** Hard cut concentrates large caller-migration work inside the class-modernisation sub-release, easily breaching the 1–3 session budget for high-fan-out classes.
- **P8 compliance.** A sub-release that leaves a class modernised but partially migrated at call sites is still usable under soft cut; under hard cut a mid-session interrupt could leave the repository non-compilable.

- **Small user base (S4).** Deprecation warnings on a small, technically sophisticated user base cost little.
- **HC2 explicitly accepted.** Phase 2c accepted dual-architecture friction as unavoidable. Soft cut honours that acceptance.

Trade-off accepted. Soft cut prolongs the period during which HC2 is felt. Deliberate cost paid to honour P3 and P8.

2.1.3 Q2 — Sub-release granularity

Decision: Mixed. Sub-release unit is not fixed; size is driven by dependencies and the realities of research-paced development.

Rationale.

- P3 constrains sub-release *behaviour* (usable state at the end), not sub-release *contents*. Fixing granularity a priori is not required.
- P8 tolerates variable granularity better than a fixed rule (natural pause points rarely coincide with class boundaries).
- P9 enforces quality via "no scope creep within the chosen sub-release," not via fixed sub-release content.
- FM1 mitigation: oversized sub-releases are a leading cause of stuck-in-the-middle; mixed granularity allows erring on the side of smaller.

2.1.4 Q3 — Internal caller migration

Decomposed into three sub-questions once Q1 (soft cut) was fixed.

Q3.1 — Default for new code after `+icnna.X` ships. Decision: (a) New code must use `+icnna`. Legacy is never a valid target for a fresh call site once its modern counterpart exists.

Rationale. Soft cut $\neq$ equivalence; without Q3.1=(a) the deprecation is toothless and soft cut degenerates into permanent duplication. P1 (data model first) and P7 (survival tier commitment) reinforce this: new consumers must be written against the modern data model or the modernisation is neutralised.

Q3.2 — Migration of existing callers. Decision: (a) opportunistic, with (b) soft overlay. No scheduled sweep sub-releases. Callers migrate when they are themselves modernised or touched for unrelated reasons. The Phase 3 roadmap may insert an opportunistic cleanup sub-release when a deprecated class's caller count drops below a threshold (approx. ≤ 3 sites — exact value deferred to Phase 3).

Rationale. Pure scheduled sweeps (pure (b)) consume whole sub-release budgets on mechanical work, violating P7. Pure opportunistic (pure (a)) risks a permanent long tail that blocks deletion. The overlay is triage, not schedule: triggered by evidence (low caller count) rather than fixed cadence. Bounds HC2 in time.

Q3.3 — Bridge code policy. Decision: (β) adapter functions in a `+icnna.compat` package, applied to modern code only. Legacy `oo/` code retains existing γ-style branching unchanged.

Evidence driving the decision. A codebase inspection conducted during the Q3.3 discussion:

- Total `isa(...,'icnna....')` usages: 275 across 172 files.

- Of these, only ∼10 are true γ-style legacy/modern bridges. The rest are standard MATLAB copy-constructor idioms (∼30+ sites) or legitimate polymorphic dispatch within the modern SNIRF object hierarchy.

- True γ in *modern* code is concentrated around `timeline`: 5–6 sites (`plot/plotTimeline`, `plot/plotStructuredData`, `op/optodeMovement_TDDR`, `op/getTrialData`, `op/+snirf/snirfNirs2timel` at two locations).

- Migration cost: ≤ 1 afternoon of focused work.

Rationale for switching from γ despite historical inertia.

1. **Cost minimum is now.** Only `timeline` is currently in true dual state; adopting β now prevents γ from proliferating across every future modernised class over the remaining ∼18 months of the v2.0.0 roadmap.

2. **Visible debt ledger** (P9 alignment). `ls +icnna/+compat/` enumerates remaining legacy bridges. A grep for scattered `isa` calls does not.

3. **Enforceable rule.** "Modern code accepts modern types; bridging happens in `+compat`" is a single sentence. γ requires per-call vigilance.

4. **Legacy carve-out** preserves historical consistency: `oo/` code is untouched, β is imposed only where there is room to apply it cleanly.

Mechanics.

```
% +icnna/+compat/toModernTimeline.m
function tm = toModernTimeline(t)
    % Accepts either legacy timeline or icnna.data.core.timeline;
    % returns icnna.data.core.timeline. Idempotent.
    if isa(t, 'icnna.data.core.timeline'), tm = t; return; end
    if isa(t, 'timeline'), tm = icnna.data.core.timeline(t); return; end
    error('icnna:compat:unsupportedType', ...);
end
```

At each γ site:

```
t = icnna.compat.toModernTimeline(t); % adapt at function boundary
% rest of function assumes modern type
```

`+icnna.compat` is documented as scheduled for deletion at v2.0.0. Each adapter carries a Doxygen-style header noting the scheduled deletion.

Naming. `+icnna.compat` preferred over `+icnna.legacy` (misleading: the package contains *modern* bridging code) or `+icnna.bridge` (collides with the Bridge design pattern). `compat` is the conventional term across software engineering.

2.1.5 Q4 — Timing of the GUI clean cut

Decision: v1.4.2 (early).

Rationale.

- HC1 (GUI coupling) is neutralised only when the cut is executed. Delaying means every class modernised in the interim breaks its GUI.

- User disruption is a one-time cost; concentrating GUI deprecation, test introduction, $\gamma \to \beta$ migration, and R2021a declaration into a single v1.4.2 release ensures the user-facing "modernisation mode begins" moment happens once.

- P2 (test-backed modernisation from day one) requires tests at v1.4.2; bundling leverages that heavy release maximally.
- No stakeholder benefits from delay.

v1.4.2 becomes the "modernisation mode begins" milestone, bundling:
- Introduction of test infrastructure (P2 requirement).
- Deprecation of all current GUIs.
- Creation of `+icnna.compat` package.
- Migration of the 5–6 existing modern γ sites to `icnna.compat.toModernTimeline`.
- R2021a declared as minimum supported MATLAB version.

2.1.6 Q5 — Definition-class lockstep

Decision: Decoupled. A class and its `*Definition` sibling are not obligated to ship together.

Rationale. P5 (flat architecture is a design task, not translation) requires the freedom to redesign the `*`/`*Definition` relationship; lockstep quietly re-imposes translation discipline. P3 keeps sub-release scope controllable; P9 allows deferring a sibling when its redesign is not yet settled.

2.1.7 E.1 — Seven operating rules

The internal transition is governed by seven operating rules derived from Q1–Q5:

1. **Soft cut per class** (Q1). Legacy `oo/@X` survives deprecated alongside `+icnna.X`.
2. **Mixed granularity** (Q2). Sub-release size dependency-driven; P3 is the constraint.
3. **Deprecation contract** (Q3.1): no new callers of legacy types once the modern counterpart exists.
4. **Opportunistic caller migration** (Q3.2), with cleanup sub-releases inserted by Phase 3 when caller count $\leq\sim 3$.
5. **Bridging via `+icnna.compat`** (Q3.3, β policy). Modern code single-type; legacy `oo/` keeps γ.
6. **v1.4.2 is the "modernisation mode begins" milestone** (Q4).
7. **`*` and `*Definition` classes decoupled** (Q5).

2.1.8 E.1 — Principle traceability

Decision	Primary principle(s)	Secondary
Q1 soft cut	P3, P8	P7
Q2 mixed granularity	P3, P9	P8
Q3.1 new code = modern only	P1, P7	—
Q3.2 opportunistic + soft overlay	P3, P7, P9	FM1 mitigation
Q3.3 β via `+icnna.compat`	P9	P2, P5
Q4 v1.4.2 clean cut	P2, P4	HC1 neutralisation
Q5 decoupled definitions	P5, P9	P3

Table 1: Principles invoked by each E.1 decision. No conflicts; where multiple principles apply, they reinforce the choice.

2.2 E.2 — Coexistence with Cantor

2.2.1 Framing

Coexistence will span an extended period (∼9+ months, likely longer at research pace). ICNNA serves current users; Cantor is built from scratch; FOE is sole developer of both. Coexistence requires a *set* of policies rather than a single decision.

Six orthogonal axes were resolved.

2.2.2 E.2.1 — Data interoperability

Decision: ICNNA and Cantor retain independent internal formats. ICNNA keeps MATLAB-based (`.mat`) storage; Cantor will likely adopt a SQLite-linked file format (decided within the Cantor project). Interchange — between the two tools and with the wider ecosystem (HomER, Cedalion, neuroDOT, MNE-NIRS, ...) — goes exclusively via **SNIRF and BIDS**. Neither project reads the other's internals.

Rationale. Cantor is clean-sheet (Phase 2b); reading legacy `.mat` would compromise that. ICNNA is bounded; adding Cantor-awareness would violate P9. Standards-first interop also serves each project's interop with the broader ecosystem at no extra engineering cost. User migration follows the same SNIRF/BIDS path any external tool would — no privileged bilateral format.

2.2.3 E.2.2 — Scope-leakage prevention

Decision: Case-by-case, with default bias that *most new features go to Cantor*. ICNNA is not in competition with Cantor; nor is it dying. It remains bounded but may accept new features when case-by-case judgement warrants.

Rationale. Phase 2c kept the door open for future ICNNA features; a strict "no new features ever" rule would contradict that. Case-by-case respects the small user base's actual needs. The Cantor-default bias makes the exception explicit: onus is on a request to justify why it belongs in ICNNA rather than Cantor.

Operational heuristic. A request is ICNNA-eligible if it is:

(a) a bug fix,
(b) survival-tier modernisation per E.1,
(c) SNIRF/BIDS compliance maintenance, or
(d) a small feature a current user needs before Cantor ships.

Everything else is Cantor-bound by default, subject to case-by-case exception.

2.2.4 E.2.3 — Shared assets

Decision: Test datasets and sample fixtures: **duplicate**. Code: **separate but referenced**. Neither project depends on the other; both are aware of each other and may cite in documentation or mathematical specifications.

Rationale. For two codebases and one developer, decoupling is worth more than deduplication. Shared assets create coupling; coupling propagates breakage. Test fixtures are stable and small — duplication is cheap over a multi-year horizon. Awareness without dependency preserves intellectual continuity without technical entanglement. A third shared-math repo was considered but deferred to Phase 3 if volume justifies.

2.2.5 E.2.4 — Developer time allocation

Decision: Phased priority with ongoing parallel work.

- **Initial phase:** ICNNA has priority until it reaches a reasonably updated, stable state (approximately: v1.4.2 shipped, test infrastructure in place, GUI deprecated, data-model modernisation well under way).
- **Subsequent phase:** Cantor becomes the primary focus, with ICNNA attention continuing in parallel.
- **Standing instruction to Claude:** flag explicitly whenever one project appears to lag behind the other, in either direction. The flag is informational; FOE decides whether to rebalance.

Rationale. ICNNA's bounded modernisation has a near-term endpoint (v1.4.2); front-loading gets the heavy release out of the way and reverts to steady-state maintenance. Fixed percentage splits are fictions at research pace (P8). The lag-detection responsibility is cheap insurance against lopsided neglect.

2.2.6 E.2.5 — Repository hygiene

Decision: Completely independent repositories. No linkage beyond README cross-references (subject to the asymmetry in E.2.6). Each project keeps its own commit schedule and conventions. Both follow the ICNNA commit-style defined in global CLAUDE.md but are not coordinated.

Paths.

- ICNNA: `C:\Users\orihuelf-admin\OneDrive\Git\ICNNA`
- Cantor: `C:\Users\orihuelf-admin\OneDrive\Git\Cantor`

No shared meta-repo, CI, or submodules.

2.2.7 E.2.6 — External communication (structural)

Decision: Asymmetric awareness.

- Cantor exhibits *higher* awareness of ICNNA. Cantor's README, documentation, and public presentation explicitly acknowledge ICNNA as predecessor, describe the relationship, point to ICNNA for current-production use, and define the migration path.
- ICNNA acts *almost blind* to Cantor. A single acknowledgement line in the ICNNA README pointing to Cantor (as successor) appears *only at/after handover*, not during the modernisation period (see refinement in E.4.2).

Rationale. Scope-discipline protection: if ICNNA documentation routinely defers features to Cantor, ICNNA's scope is re-opened for debate every time a user asks. Successor projects own the transition narrative — it is Cantor's job to situate itself relative to what exists. Asymmetric awareness also reduces documentation churn on the ICNNA side as Cantor evolves.

2.3 E.3 — Handover to Cantor

2.3.1 Framing

Handover is the moment at which Cantor becomes the primary recommended tool and ICNNA transitions out of "actively modernising" role. A structural finding guided E.3 overall: **ICNNA is not scheduled to die.** Phase 2c kept the door open; E.3.3 and E.3.4 confirm ICNNA remains a live, maintained project post-handover. Handover changes ICNNA's *role*, not its lifecycle.

Six axes were resolved.

2.3.2 E.3.1 — Handover trigger

Decision: Composite: Cantor v1.0.0 released **and** Cantor reaches feature parity with ICNNA (verified against a checklist). Usage-based validation component (a successful end-to-end user migration) is *not* part of the trigger — FOE handles user-side validation personally given the small community. Operational details (the feature-parity checklist itself) deferred to Phase 3.

Rationale. v1.0.0 alone is insufficient (a self-declaration does not guarantee parity); feature parity alone is insufficient (without v1.0.0 it is a moving target). Pairing freezes the question. Usage thresholds are unnecessary given direct personal shepherding.

2.3.3 E.3.2 — User migration path

Decision: Personal guidance (FOE-shepherded) with minimal written support; no forced migration. Data migration follows E.2.1 (SNIRF/BIDS). FOE shepherds each user individually when that user is ready. A short migration guide accompanies the v1.0.0/feature-parity announcement. No time-boxed migration window. Users may decline to migrate indefinitely.

Rationale. Written docs scale; for 5–10 users, one-to-one is higher-quality and tractable. No forced migration preserves trust; since ICNNA stays alive (E.3.3), nothing compels users to move.

2.3.4 E.3.3 — Post-handover repository status

Decision: Maintained. ICNNA remains a live, maintained project: repository public and active; bug fixes accepted; new-feature work possible (see E.3.4); documentation updated.

Rationale. Phase 2c stance preserved ("v2.0.0 is the endpoint of the modernisation plan, not the end of ICNNA's journey"). The loyal small user base is retained; a frozen-or-archived posture would push users off prematurely, contradicting E.3.2. ICNNA's MATLAB-native niche is durable (S5 from Phase 2c).

2.3.5 E.3.4 — Post-v2.0.0 release posture

Decision: Occasional feature releases permitted. v2.0.x for maintenance; v2.1.x, v2.2.x, etc. when a new feature is warranted and FOE has time and interest. No cadence commitment.

Rationale. Consistent with E.3.3 (a "maintained" project that can never grow new features is frozen in practice). E.2.2 case-by-case discipline continues to apply post-handover. "Occasional" means zero-or-more per year; no release is ever owed.

2.3.6 E.3.5 — Long-tail support triage

Decision: Category-based triage.

Category	Policy
Critical data-integrity / security	Patch release promptly, no time bound on commitment
Standards drift (SNIRF/BIDS spec update)	Best effort, non-urgent, bundled with next convenient release
MATLAB version break	Case-by-case; may resolve by declaring the MATLAB version unsupported rather than patching

Rationale. Matches urgency to category. Preserves FOE's ability to say no — essential for a single developer maintaining two projects.

Mitigation carried forward (R.E3.5.1). The softness of "best effort" and "case-by-case" can leave a user stranded on an unsupported MATLAB version with unclear recourse. Mitigation: clear MATLAB-version support statement at handover, setting explicit expectations (agreed).

2.3.7 E.3.6 — Intellectual asset preservation

Context (institutional). ICNNA's intellectual assets are distributed:

Institution	ICNNA-related intellectual assets
Imperial College London (IC)	Original ICNNA framework (developed at IC)
INAOE (Mexico)	MENA Gen 2, MENA Gen 3, ontoNIRS (at INAOE)
University of Birmingham (UoB)	Overprocessing, pipeline optimisation, Cantor (at UoB)

FOE maintains ICNNA personally under peer-to-peer agreements with the institutions where the work was developed. These agreements permit continued maintenance but do not transfer IP ownership.

Decision for E.3.6 proper.

- Intellectual assets (papers, theses, formal specifications, LaTeX documents) stay where authored. They remain in ICNNA's `doc/` structure. No forced relocation to Cantor.
- Cantor may cite ICNNA intellectual assets (consistent with E.2.6).
- Cantor's relationship to ICNNA intellectual content is, in the normal case, one of **honest academic citation and referencing**, not IP reuse. Standard scholarly practice covers this. Specific cases of genuine reuse (if any) are handled individually at the time under the relevant institutional agreements — not a general project risk.

Recommended artefact: IP ledger. Create `doc/ip_ledger.md` (or `.tex`) in ICNNA mapping major features/modules/specifications to owning institutions. Benefits: makes the ownership map explicit; provides paper trail; lets Cantor cite correctly. **Creation deferred to Phase 3 as a collaborative pass.**

2.3.8 E.3 — Policy summary

1. Trigger: composite of Cantor v1.0.0 *and* feature parity.
2. Migration: FOE-shepherded, SNIRF/BIDS mechanics, no forced timeline.
3. Post-handover status: maintained (not frozen, not archived).
4. Post-v2.0.0 releases: maintenance plus occasional features; no cadence commitment.
5. Long-tail support: category-based triage.
6. IP preservation: assets stay where authored; IP ledger artefact recommended (deferred).

2.4 E.4 — User communication strategy

2.4.1 Framing

Three temporally distinct periods require distinct messaging:

1. **Modernisation period** — now through v2.0.0.
2. **Handover transition** — at the Cantor v1.0.0 + feature-parity moment.
3. **Post-handover steady state** — ICNNA maintained, Cantor primary.

Five axes were resolved.

2.4.2 E.4.1 — GUI deprecation messaging at v1.4.2

Decision: Matter-of-fact, forward-looking release-notes posture. Short explanation: the GUI is deprecated to permit clean architectural modernisation; a new GUI is planned late in the v2.0.0 roadmap. Substitute: command-line/script workflows documented with at least one worked example in `doc/`. Advance notice: mentioned in v1.4.1/v1.4.1-beta release notes if practical.

Rationale. Honesty over apology; apologetic language invites prolonged debate. Forward-looking framing preserves user confidence. Worked examples reduce support burden.

2.4.3 E.4.2 — README posture during modernisation

Decision: ICNNA's public communications act *as if unaware of Cantor* during the modernisation period. No mention of Cantor in ICNNA's README, documentation, or user-facing materials until handover approaches.

This refines E.2.6. The "single acknowledgement line pointing to Cantor" is *deferred* until handover, not present during modernisation.

Rationale. Cantor is not ready; a premature reference invites users to try an unready tool and return disappointed — worst-of-both outcome. ICNNA is usable *now* and its README should sell the current product, not hedge. Absence of reference = zero maintenance cost on ICNNA documentation as Cantor evolves (P9). Preserves Cantor's announcement freedom.

README content during modernisation. What ICNNA is; current version and state; installation; command-line/script usage posture; pointer to `ROADMAP.md`; license, citation, contact, acknowledgements. **No Cantor reference.**

2.4.4 E.4.3 — Handover-moment announcement

Decision: Deferred. The form, channels, and timing are undecided and will be revisited closer to handover.

Rationale. Depends on context unavailable now (both projects' state, FOE's institutional context at the time, community needs then). Decision cost is low at the moment. No operational dependency.

2.4.5 E.4.4 — Post-handover messaging

Decision: Neutral factual posture. README updated *once* to reflect new role (e.g. "ICNNA remains maintained for users with ongoing MATLAB-based workflows; Cantor is the recommended choice for new work"). The single acknowledgement line from E.2.6 lands here. Release notes continue normally for patch/maintenance releases. No anti-Cantor messaging; no pro-Cantor advocacy. Neutral factual acknowledgement only.

Rationale. Consistent with E.3.3 (maintained). Users of a maintained tool want stability; drama would unsettle those who chose to stay. Respects Cantor's ownership of the transition narrative (E.2.6).

2.4.6 E.4.5 — CHANGELOG conventions

Decision: Adopt `CHANGELOG.md` at the repository root (Keep-a-Changelog conventions). Both ICNNA and Cantor will have one, each authored independently.

- **Location:** `CHANGELOG.md` at the repository root — standard practice, first-class metadata file alongside `README.md` and `LICENSE`. Not under `doc/`.
- **Seed:** `doc/ICNNA-Version.log` is the seed for ICNNA's CHANGELOG. Content migrated; style modernised to Keep-a-Changelog sections (Added / Changed / Deprecated /

Removed / Fixed / Security). **Full migration, per the transparency principle** (Q.2; see §3.E below and global CLAUDE.md §9).

- **Historical archive retained:** `doc/ICNNA-Version.log` remains in place with a forward pointer to `CHANGELOG.md` added at its top. No deletion — preserves behavioural traceability per the "never overwrite" principle.
- **Cross-references:** major CHANGELOG entries link to the corresponding `claude/minutes/` minute by symbolic reference (see §3.E, Q.5 resolution).
- **Commit-level style** continues to follow global CLAUDE.md (feat/fix/refactor/test/docs/chore).

Implementation of CHANGELOG seeding is a Phase 3 task.

2.4.7 E.4 — Policy summary

1. GUI deprecation communicated matter-of-factly (E.4.1).
2. ICNNA silent on Cantor during modernisation (E.4.2) — refinement of E.2.6.
3. Handover-moment announcement deferred (E.4.3).
4. Post-handover ICNNA neutral (E.4.4).
5. `CHANGELOG.md` at repo root, seeded from `doc/ICNNA-Version.log` (E.4.5).

3 Step F — Registry of Open Items

Step F records the state of what is *not* settled at the end of Phase 2, so that Phase 3 and subsequent sessions can resume without re-deriving context. Items are prefixed: `R.*` (risk), `U.*` (unknown), `D.*` (deferred decision), `Q.*` (open question). The registry is ongoing.

3.1 A. Accepted risks (acknowledged, tolerated, not technically mitigated)

ID	Description	Disposition
R.FM1	Stuck-in-the-middle: modernisation stalls leaving a worse dual-architecture state	Procedural mitigation via P3; no technical mitigation
R.FM4	Standards drift (SNIRF/BIDS spec changes)	Deferred; address if/when standards change
R.HC2	Dual-architecture friction during transition	Bounded by β + `+icnna.compat`; not eliminated
R.HC5	Research-priority interrupts	Mitigated by P8 structurally
R.L4	No AI integration in ICNNA	Accepted; AI goes to Cantor
R.L5	MathWorks vendor lock-in	Accepted

3.2 B. Unmitigated risks carried forward

ID	Description	Mitigation path
R.E3.1.1	Feature-parity definition deferred; if never settled, handover cannot fire	Phase 3 to address at appropriate moment
R.E3.5.1	Soft long-tail support commitments may strand users on unsupported MATLAB versions	Clear MATLAB-version support statement at handover
R.F1	Developer-time policy may degenerate into "only the active project gets attention" under interrupt pressure	Claude's lag-detection standing responsibility; FOE's parallel-work discipline. Residual structural risk
R.F2	Single-developer bus factor (FOE sole developer of both projects)	Partial mitigation by sequencing: front-load ICNNA to reach stable state, then focus on Cantor without ICNNA half-finished. Structural
R.F3	OneDrive sync conflicts across devices on hot files	Portable-context rule (global CLAUDE.md §8) and git discipline. Residual risk accepted

3.3 C. Deferred decisions (to Phase 3 or later)

ID	Item	Raised in
D.1	Q3.2 cleanup trigger threshold (≤ 3? ≤ 5?)	E.1
D.2	Deprecation warning mechanism specifics for legacy classes	E.1
D.3	Doxygen-style header convention for `+icnna.compat` members	E.1
D.4	Exact trigger milestone for ICNNA→Cantor priority shift	E.2.4
D.5	Feature-parity checklist for E.3.1 handover trigger	E.3.1
D.6	Migration-guide skeleton (SNIRF/BIDS export → Cantor import)	E.3.2
D.7	IP ledger artefact (`doc/ip_ledger.md`) — collaborative Phase 3 pass	E.3.6
D.8	MATLAB-version support statement phrasing	E.3.5
D.9	Handover-moment announcement form, channels, timing	E.4.3
D.10	Exact wording of post-handover README Cantor acknowledgement line	E.4.4
D.11	`CHANGELOG.md` seeding from `doc/ICNNA-Version.log` (full migration, per §9)	E.4.5
D.12	Worked-example content for command-line workflows accompanying v1.4.2 GUI deprecation	E.4.1
D.13	GUI v2.0.0 design	Phase 2c
D.14	BIDS compatibility implementation details	Phase 2c
D.15	Test infrastructure framework choice and structure (presumably `matlab.unittest`)	P2, v1.4.2
D.16	Scheduling of `@rawData_Snirf` special case (HC6) — may jump queue	Phase 2c

3.4 D. Unknowns (need observation, not decision)

ID	Item	Impact
U.1	Cantor development timeline (> 9 months est., research pace)	Length of coexistence
U.2	Actual user response to v1.4.2 GUI deprecation	Post-v1.4.2 scheduling and messaging
U.3	Standards evolution (SNIRF, BIDS) during coexistence	Maintenance load; see R.FM4
U.4	FOE's future institutional affiliation during the horizon	FOE handles personally
U.5	Cantor's final technical stack details (file format, build, packaging)	E.2.1 interop edge cases
U.6	Whether any ICNNA user declines to migrate indefinitely	Post-handover ICNNA activity level

3.5 E. Open questions resolved during this session

All open questions raised in the draft registry were addressed.

ID	Question	Resolution
Q.1	R2021a minimum appropriate for all modernised features?	Stick to R2021a. If a later-version feature is ever needed, Claude must warn FOE first and attempt to circumvent; raise minimum only if genuinely impossible, as explicit decision
Q.2	CHANGELOG seeding: any `doc/ICNNA-Version.log` content that should not migrate publicly?	No. Full migration. Full disclosure is FOE's default policy across all public projects; Claude must warn-and-ask on specific material of concern (codified in global CLAUDE.md §9)
Q.3	Package name: `+icnna.compat` vs `legacy` vs `bridge`?	`+icnna.compat` — conventional across software engineering; accurately describes bridging purpose; avoids design-pattern collision
Q.4	IP ledger: FOE alone or collaborative Phase 3 pass?	Collaborative pass in Phase 3
Q.5	Citation format for `claude/` minutes from CHANGELOG / release notes — most future-proof?	Symbolic reference by document title + section anchor (e.g. "see Phase 2d minute, §E.1"). Survives folder reorganisation, repo relocation, format changes. Permalinks permitted as supplementary only

3.6 F. New policies codified during Phase 2 Session 4

1. **Context and memory portability** (global CLAUDE.md §8, added 14-Apr-2026). Project-local `.\claude\` folder is the canonical source of truth for session context, decisions, and memory-equivalent state. Auto-memory is a device-local cache, never the only place a non-trivial fact lives. Includes a recommended subfolder structure (`memory/`, `minutes/`, `prompts/`, `state/`, `media/`) applicable across projects.

2. **Transparency and disclosure** (global CLAUDE.md §9, added 14-Apr-2026). Full-disclosure default; warn-before-filter exception handling; applies across projects unless ex-

plicitly overridden.

3. **Proactive project-lag flagging** (project memory, 14-Apr-2026). Claude flags when ICNNA or Cantor appears to lag behind the other, informationally only; FOE decides whether to rebalance.

4. **R2021a minimum standing policy** (project memory, 14-Apr-2026). Claude warns before using any MATLAB feature introduced after R2021a, attempts circumvention; minimum raised only as explicit decision.

5. **Stable-title minute citation convention** (project convention, 14-Apr-2026). Every `claude/minutes/` minute has a stable human-readable title used as the anchor in citations.

4 Step G — Session Documentation and Folder Reorganisation

Step G produced this minute, compiled it to PDF, and reorganised the `claude/` folder from its previous flat layout to a future-proof portable structure.

4.1 G.1 — This minute

This document (`Phase2d_minute.tex`) is the Phase 2 closing minute. It consolidates Steps E.1–E.4 and F into a single coherent record. Per FOE's instruction, length was not pre-constrained; clarity and 10–15 year traceability were the priorities. Interim per-step state notes (`Phase2d_E1_state.md`, . . . , `Phase2d_F_state.md`) are retained under `claude/state/` as primary-source records of the progression, not deleted.

4.2 G.2 — Build and cleanup

Compiled with two passes of `pdflatex`. Intermediate auxiliary files (`.aux`, `.log`, `.out`, `.synctex.gz`, `.toc`) removed after successful build. Only `.tex` source and `.pdf` output retained. The `\newcommand{\mediaDir` convention is established in the preamble of this minute for future use; the `claude/media/` folder is created as a placeholder.

4.3 G.3 — Folder reorganisation

4.3.1 Rationale

The pre-existing `claude/` folder was flat and mixed concerns: session prompts, minutes, build artefacts, exploratory scratch prompts, and (pending) state notes. It had no dedicated location for portable project memory (which per global CLAUDE.md §8 must reside in project-local `claude/`). With ongoing Phase 3 work, the folder would grow — more minutes, more state notes, eventually media and bibliography — and the flat layout would become unnavigable.

4.3.2 New layout

```
claude/
  README.md # folder index + conventions
  .gitignore # ignores TeX build artefacts

  memory/ # PORTABLE PROJECT MEMORY (canonical)
    MEMORY.md # index
    user_profile.md
    project_*.md
    feedback_*.md
    ...
```

```
minutes/ # SESSION MINUTES
  Phase1_minute.tex / .pdf
  Phase2a_minute.tex / .pdf
  Phase2b_minute.tex / .pdf
  Phase2c_minute.tex / .pdf
  Phase2d_minute.tex / .pdf # this document

prompts/ # SESSION INPUT PROMPTS
  Phase1_context_prompt.txt
  Phase2a_context_prompt.txt
  Phase2b_context_prompt.txt
  Phase2c_context_prompt.txt
  Phase3_context_prompt.txt # draft handoff
  legacy/ # early-session scratch prompts
    ClaudeCode_promp0001.txt
    ClaudeCode_promp0001b.txt
    answers0001b.txt

state/ # INTERIM SESSION STATE NOTES
  Phase2d_E1_state.md
  Phase2d_E2_state.md
  Phase2d_E3_state.md
  Phase2d_E4_state.md
  Phase2d_F_state.md

media/ # IMAGES / BIB (future use)
```

4.3.3 Design points

1. `memory/` **is canonical.** Memory files are written to `claude/memory/` first (and mirrored to device-local auto-memory as a convenience cache). This operationalises global CLAUDE.md §8.

2. **Auto-memory files copied into** `claude/memory/` during the reorganisation, establishing the portable canonical record.

3. `prompts/legacy/` separates early exploratory prompts from formal phase context prompts.

4. `state/` **preserved, not deleted.** Per the "never overwrite" principle: state notes are primary-source records of the progression, retained alongside the consolidated minute.

5. `README.md` at folder root documents the structure, the memory-canonical rule, the citation convention, and the lifecycle of each subfolder's contents.

6. `.gitignore` excludes TeX build artefacts (`*.aux`, `*.log`, `*.out`, `*.synctex.gz`, `*.toc`, `*.fls`, `*.fdb_latexmk`).

7. **No per-phase subfolders** (e.g. `minutes/Phase1/`). File names encode the phase; flat-within-category is easier to scan. Revisit only when any subfolder exceeds ∼20 items.

8. **Trade-off rejected:** committing only `.tex` sources without built PDFs would prevent readers without a TeX installation from accessing the minutes, contradicting the full-disclosure principle (§9).

4.3.4 Global CLAUDE.md update

Global CLAUDE.md §8.4 extended on 14-Apr-2026 to document the recommended subfolder structure as a default for all of FOE's projects (flexible: subfolders only created when populated; project-level CLAUDE.md may refine).

4.3.5 Execution

- `git mv` used for file moves to preserve rename tracking.
- Searched for hard-coded path references in prior minute sources before moving; none material to the reorganisation found.
- Reorganisation applied as a single atomic commit after this minute was produced.

4.4 G.4 — Draft Phase 3 context prompt

A draft Phase 3 context prompt has been produced at `claude/prompts/Phase3_context_prompt.txt`, following the style of previous phase prompts. It serves as a fallback and as input to the prompt FOE will prepare independently. At the start of Phase 3, Claude must first ask FOE for the prepared prompt; FOE's prompt takes precedence, and the draft is used only if the prepared prompt is not immediately available. This instruction is recorded in project memory.

5 Phase 2 Closing Statement

Phase 2 of the ICNNA architectural refactoring — the architectural decision phase — is **formally closed** with this minute.

5.1 What Phase 2 delivered

1. **The two-project split** (Phase 2b). Cantor created as the clean-sheet successor (C++23/wxWidgets); ICNNA scoped to a bounded modernisation in MATLAB.
2. **Nine design principles** (Phase 2c, P1–P9) governing the bounded ICNNA update.
3. **Seven operating rules** (Phase 2d, §2.1) governing the internal transition from the current dual state to the v2.0.0 endpoint.
4. **Six coexistence policies** (Phase 2d, §2.2) governing ICNNA/Cantor parallel operation.
5. **Six handover policies** (Phase 2d, §2.3) governing the transition to Cantor as primary tool, with ICNNA remaining maintained.
6. **Five communication policies** (Phase 2d, §2.4) across three temporal periods.
7. **An open-items registry** (Phase 2d, §3) of ∼30 items across five categories.
8. **Five new policies codified** (Phase 2d, §3.F), two at global CLAUDE.md level.
9. **A reorganised and documented `claude/` folder** (Phase 2d, §4) that is portable, canonical for project memory, and future-proof.

5.2 What Phase 2 did not decide

Phase 2 deliberately did not descend into:

- The concrete per-version content of v1.4.2, v1.4.3, . . . , v2.0.0 — this is Phase 3.
- Specific class-level modernisation order within the data-model slice — Phase 3.
- The feature-parity checklist for the handover trigger — Phase 3 or Cantor-side.
- Detailed specification of GUI v2.0.0 — deferred to when the modern architecture is sufficiently complete.

5.3 Sufficiency for a cold start

Phase 2 closes with the explicit intent that Phase 3 can begin from a cold context on any device, reading only the canonical artefacts under `claude/`. This is both a design goal (global CLAUDE.md §8) and an operational validation: the next session will test whether the portability principle actually holds. Any gap surfaced at that moment is itself valuable — it identifies what the canonical record is missing and must be fixed before Phase 3 proceeds.

6 Hand-off to Phase 3

Phase 3 objective: produce the concrete roadmap from ICNNA v1.4.1-beta (current) through v1.4.2 (the "modernisation mode begins" milestone) through subsequent sub-releases to v2.0.0.

Phase 3 inputs.

- All Phase 2 minutes, especially this one and `Phase2c_minute.pdf` (principles).
- The open-items registry (this minute, §3; Phase 2d F state note).
- The sixteen deferred decisions (D.1–D.16) are concrete roadmap candidates.
- The five unmitigated risks (B section) to be monitored.
- The seven operating rules (E.1) constraining how sub-releases are shaped.
- v1.4.2 milestone content already identified: test infrastructure introduction; GUI deprecation; `+icnna.compat` creation and $\gamma \to \beta$ migration of 5–6 sites; R2021a declared minimum.

Phase 3 starting action. FOE will provide a prepared Phase 3 context prompt at session start. Claude must ask for it at the very beginning of the next session. The draft prompt at `claude/prompts/Phase3_context_prompt.txt` is a fallback only. FOE's prompt takes precedence.

Session-splitting rationale. Phase 3 begins in a *new* Claude session (hard split) rather than continuing in this one. Reasons: (i) validates the canonical-context design; (ii) matches the 10–15 year traceability principle (a future reader starts cold); (iii) avoids contamination by this session's exploratory conversational artefacts; (iv) gives Phase 3 a full context-window budget.

7 Decisions Recorded (Combined Table)

Area	Decision
E.1 — Internal transition	
Q1 coexistence	Soft cut: legacy `oo/@X` survives deprecated alongside modern `+icnna.X`
Q2 granularity	Mixed; dependency-driven, not fixed
Q3.1 new code	Must use `+icnna` once modern counterpart exists
Q3.2 caller migration	Opportunistic, with Phase 3 cleanup sub-release when caller count $\leq\sim 3$
Q3.3 bridging	β policy via `+icnna.compat` package; modern code single-type; legacy `oo/` retains γ
Q4 GUI cut	v1.4.2 — bundled with test infra, $\gamma \to \beta$ migration, R2021a declaration

Area	Decision
Q5 definition lockstep	Decoupled; no 1-to-1 pairing required
E.2 — Coexistence	
E.2.1 interop	SNIRF/BIDS only; no reading of each other's internals
E.2.2 scope	Case-by-case with Cantor default; ICNNA accepts narrow-scope only
E.2.3 assets	Datasets duplicated; code separate but referenced
E.2.4 time	Phased (ICNNA first, Cantor primary later); parallel throughout; Claude flags lag
E.2.5 repos	Fully independent; no git linkage
E.2.6 awareness	Asymmetric: Cantor aware of ICNNA; ICNNA blind to Cantor (refined in E.4.2)
E.3 — Handover	
E.3.1 trigger	Cantor v1.0.0 + feature parity (composite); user-validation handled personally by FOE
E.3.2 migration	FOE-shepherded; SNIRF/BIDS; no forced timeline
E.3.3 post-handover status	Maintained (not frozen, not archived)
E.3.4 post-v2.0.0 releases	Maintenance + occasional features; no cadence commitment
E.3.5 long-tail support	Category-based triage
E.3.6 IP preservation	Assets stay where authored; IP ledger artefact recommended (deferred Phase 3)
E.4 — Communication	
E.4.1 v1.4.2 notes	Matter-of-fact, forward-looking, CLI substitutes documented
E.4.2 README modernisation	Silent on Cantor; refines E.2.6
E.4.3 handover announcement	Deferred to closer to the moment
E.4.4 post-handover messaging	Neutral; one-time README update; no advocacy
E.4.5 CHANGELOG	`CHANGELOG.md` at repo root, Keep-a-Changelog, seeded from `doc/ICNNA-Version.log` (full migration, §9 principle); original log retained with forward pointer
F — Policies codified	
Global §8	Context/memory portability — `claude/` canonical; recommended subfolder structure
Global §9	Transparency and disclosure — full disclosure default; warn-before-filter
Project memory	Proactive project-lag flagging (ICNNA/Cantor)
Project memory	R2021a minimum standing policy
Project convention	Stable-title minute citation ("see Phase 2d minute, §E.1")
G — Folder reorganisation	

Area	Decision
Layout	`claude/{memory,minutes,prompts,state,media}/` with `README.md` and `.gitignore` at root
Canonical memory	`claude/memory/` supersedes device-local auto-memory as source of truth
State notes	Retained alongside minutes per "never overwrite"
Build policy	Aux files removed post-compile; only `.tex`/`.pdf` tracked

8 Documents and Artefacts Referenced

1. **Phase 2 predecessors.** `claude/minutes/Phase1_minute.tex`, `Phase2a_minute.tex`, `Phase2b_minute.tex`, `Phase2c_minute.tex`.
2. **This minute's source state notes.** `claude/state/Phase2d_E1_state.md`, `Phase2d_E2_state.md`, `Phase2d_E3_state.md`, `Phase2d_E4_state.md`, `Phase2d_F_state.md`.
3. **Global and project rules.** Global `CLAUDE.md` (as updated 14-Apr-2026, §§8–9); project `CLAUDE.md`.
4. **Canonical memory.** `claude/memory/MEMORY.md` and referenced files within.
5. **Cantor project location.** `C:\Users\orihuelf-admin\OneDrive\Git\Cantor`.
6. **Phase 3 handoff.** `claude/prompts/Phase3_context_prompt.txt` (draft).